01 — Introduction to PHP

What PHP is

PHP is a language that runs **on the server**. The browser asks for a page, PHP runs, produces HTML, and only that HTML is sent back. The visitor never sees your PHP code.

```
Browser  --request-->  Web server + PHP  --runs page.php-->  HTML
Browser  <--HTML-----  Web server
```

A few facts worth knowing:

- Created in 1994 by Rasmus Lerdorf ("Personal Home Page", now "PHP: Hypertext Preprocessor").
- Powers a large share of the web: WordPress, Drupal, Magento, Laravel and Symfony apps.
- PHP 7 (2015) made it about twice as fast. PHP 8 (2020+) added modern features: named arguments, `match`, enums, readonly properties, union types.
- Dynamically typed, like Python, with optional type declarations, a bit like Java.

Configuring the engine

PHP reads its settings from a file called `php.ini`.

```
php --ini        # shows which php.ini is loaded
php -m           # lists loaded extensions
php -i           # all settings (long!)
```

In a web page, `<?php phpinfo(); ?>` shows the same information as a page. **Never leave it on a public site**: it reveals your configuration.

Settings you'll meet most often:

Setting	Meaning	Dev value
`display_errors`	Show errors in the page	`On` (never in production)
`error_reporting`	Which errors are reported	`E_ALL`
`upload_max_filesize`	Max size of an uploaded file	`2M` by default
`memory_limit`	Max memory per script	`128M` by default
`date.timezone`	Default timezone	`Europe/Paris`

You can also change some settings from code: `ini_set('display_errors', '1');`

PHP inside HTML

A `.php` file is HTML by default. PHP code goes between `<?php` and `?>`.

```
<!DOCTYPE html>
<html>
<body>
    <h1><?php echo "Hello"; ?></h1>
    <p>Today is <?= date('l') ?></p>   <!-- <?= ... ?> is a shortcut for echo -->
</body>
</html>
```

A file containing **only** PHP (classes, scripts) starts with `<?php` and should **not** end with `?>`: this avoids sending stray whitespace by accident.

Instructions end with `;`. Comments: `// one line`, `# one line`, `/* several lines */`.

Variables

Variables start with `$`. No declaration, no type needed.

```php
$name = "Alice";
$age = 32;
$price = 19.99;
$isAdmin = false;
$nothing = null;
```

The four scalar types: `string`, `int`, `float`, `bool`. Plus `array`, `object` and `null`.

```php
var_dump($age);       // int(32)  -> type AND value, your best debugging friend
echo gettype($price); // double (historical name for float)
```

Strings

```php
$first = 'Alice';
echo 'Hello $first';      // Hello $first   (single quotes: no interpolation)
echo "Hello $first";      // Hello Alice    (double quotes: variables replaced)
echo "Hello {$first}!";   // braces: safer when text follows
echo 'Hello ' . $first;   // . is the concatenation operator (not +)
```

Constants

```php
const VAT_RATE = 0.2;           // preferred, at top level or in classes
define('SITE_NAME', 'My shop'); // older style, works anywhere at runtime

echo VAT_RATE;                  // no $
```

Built-in constants: `PHP_VERSION`, `PHP_EOL` (line break), `PHP_INT_MAX`.

Type juggling and casting

PHP converts types automatically when needed. That's handy and sometimes surprising.

```php
var_dump("5" + 3);      // int(8)    the string is converted to a number
var_dump("5" . 3);      // string(2) "53"
var_dump(10 / 4);       // float(2.5)
var_dump(intdiv(10, 4)); // int(2)
```

Explicit conversion (casting):

```php
$n = (int) "42abc";   // 42
$f = (float) "3.14";  // 3.14
$s = (string) 12;     // "12"
$b = (bool) "";       // false
```

Values that are `false` when converted to bool: `false`, `0`, `0.0`, `""`, `"0"`, `[]`, `null`. Everything else is `true`.

`==` vs `===`

```php
var_dump(0 == "0");     // true  : values compared after conversion
var_dump(0 === "0");    // false : value AND type must match
var_dump(null == false); // true
```

Rule: use `===` and `!==` by default. Use `==` only when you mean it.

Strict types

Add this at the very top of a file to forbid automatic conversion when calling typed functions:

```php
<?php
declare(strict_types=1);
```

We'll see typed functions in module 02.

Now do the exercises.

01 — Exercises

Create your files in a working folder of your own (for example `my-work/01/`).

1.1 — Explore your configuration

1. Run `php --ini` and find the path of the loaded `php.ini`.
2. Run `php -m` and check that `pdo_sqlite` is in the list.
3. Create `info.php` containing `<?php phpinfo();`, start `php -S localhost:8000` in that folder and open http://localhost:8000/info.php. Find the values of `display_errors` and `upload_max_filesize`.

No solution file: compare your answers with your neighbour or the trainer.

1.2 — Business card page

Create `card.php`, a web page that displays a business card.

- Store your first name, last name, job and year of birth in variables.
- Store the current year with `date('Y')`.
- Display: the full name in an `<h1>`, the job, and the age (computed).
- Use both `<?php echo ... ?>` and the short form `<?= ... ?>` at least once.

Run it with `php -S localhost:8000` and open http://localhost:8000/card.php.

1.3 — Type detective

Create `types.php`, a command-line script (`php types.php`).

1. Predict, **on paper first**, the result of each line, then run with `var_dump()` to check:

```php
"10" + 5
"10" . 5
"3 apples" + 2      // PHP 8 gives a warning here: read it
7 / 2
intdiv(7, 2)
7 % 2
(int) "12.9"
(bool) "0"
(bool) "false"
0 == ""
0 === ""
null ?? "default"
```

2. Define a constant `VAT_RATE` (0.2) and a variable `$priceExclTax` (49.90). Display the price including tax, rounded to 2 decimals with `round()`.

3. Display your result with `number_format($value, 2, ',', ' ')` to get the French format (`59,88`).